

当用点运算符访问一个成员函数时，通常我们是想（效果也确实是）调用该函数。我们使用调用运算符（`()`）来调用一个函数。调用运算符是一对圆括号，里面放置实参（`argument`）列表（可能为空）。成员函数 `isbn` 并不接受参数。因此

```
item1.isbn()
```

调用名为 `item1` 的对象的成员函数 `isbn`，此函数返回 `item1` 中保存的 ISBN 书号。

24

在这个 `if` 条件中，相等运算符的右侧运算对象也是这样执行的——它返回保存在 `item2` 中的 ISBN 书号。如果 ISBN 相同，条件为真，否则为假。

1.5.2 节练习

练习 1.23: 编写程序，读取多条销售记录，并统计每个 ISBN（每本书）有几条销售记录。

练习 1.24: 输入表示多个 ISBN 的多条销售记录来测试上一个程序，每个 ISBN 的记录应该聚在一起。

1.6 书店程序

现在我们已经准备好完成书店程序了。我们需要从一个文件中读取销售记录，生成每本书的销售报告，显示售出册数、总销售额和平均售价。我们假定每个 ISBN 书号的所有销售记录在文件中是聚在一起保存的。

我们的程序会将每个 ISBN 的所有数据合并起来，存入名为 `total` 的变量中。我们使用另一个名为 `trans` 的变量保存读取的每条销售记录。如果 `trans` 和 `total` 指向相同的 ISBN，我们会更新 `total` 的值。否则，我们会打印 `total` 的值，并将其重置为刚刚读取的数据（`trans`）：

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item total; // 保存下一条交易记录的变量
    // 读入第一条交易记录，并确保有数据可以处理
    if (std::cin >> total) {
        Sales_item trans; // 保存和的变量
        // 读入并处理剩余交易记录
        while (std::cin >> trans) {
            // 如果我们仍在处理相同的书
            if (total.isbn() == trans.isbn())
                total += trans; // 更新总销售额
            else {
                // 打印前一本书的结果
                std::cout << total << std::endl;
                total = trans; // total 现在表示下一本书的销售额
            }
        }
        std::cout << total << std::endl; // 打印最后一本书的结果
    } else {
```

25

```
//没有输入！警告读者
std::cerr << "No data?!" << std::endl;
return -1; // 表示失败
}
return 0;
}
```

这是到目前为止我们看到的最复杂的程序了，但它所使用的都是我们已经见过的语言特性。

与往常一样，首先包含要使用的头文件：来自标准库的 `iostream` 和自己定义的 `Sales_item.h`。在 `main` 中，我们定义了一个名为 `total` 的变量，用来保存一个给定的 ISBN 的数据之和。我们首先读取第一条销售记录，存入 `total` 中，并检测这次读取操作是否成功。如果读取失败，则意味着没有任何销售记录，于是直接跳到最外层的 `else` 分支，打印一条警告信息，告诉用户没有输入。

假定已经成功读取了一条销售记录，我们继续执行最外层 `if` 之后的语句块。这个语句块首先定义一个名为 `trans` 的对象，它保存读取的销售记录。接下来的 `while` 语句将读取剩下的所有销售记录。与我们之前的程序一样，`while` 条件是一个从标准输入读取值的操作。在本例中，我们读取一个 `Sales_item` 对象，存入 `trans` 中。只要读取成功，就执行 `while` 循环体。

`while` 的循环体是一个单独的 `if` 语句，它检查 ISBN 是否相等。如果相等，使用复合赋值运算符将 `trans` 加到 `total` 中。如果 ISBN 不等，我们打印保存在 `total` 中的值，并将其重置为 `trans` 的值。在执行完 `if` 语句后，返回到 `while` 的循环条件，读取下一条销售记录，如此反复，直至所有销售记录都处理完。

当 `while` 语句终止时，`total` 保存着文件中最后一个 ISBN 的数据。我们在语句块的最后一条语句中打印这最后一个 ISBN 的 `total` 值，至此最外层 `if` 语句就结束了。

1.6 节练习

练习 1.25: 借助网站上的 `Sales_item.h` 头文件，编译并运行本节给出的书店程序。

小结

26

本章介绍了足够多的 C++ 语言的知识，以使你能够编译、运行简单的 C++ 程序。我们看到了如何定义一个 main 函数，它是操作系统执行你的程序的调用入口。我们还看到了如何定义变量，如何进行输入输出，以及如何编写 if、for 和 while 语句。本章最后介绍了 C++ 中最基本的特性——类。在本章中，我们看到了，对于其他人定义的一个类，我们应该如何创建、使用其对象。在后续章节中，我们将介绍如何定义自己的类。

术语表

参数 (实参, argument) 向函数传递的值。

赋值 (assignment) 抹去一个对象的当前值，用一个新值取代之。

程序块 (block) 零条或多条语句的序列，用花括号包围。

缓冲区 (buffer) 一个存储区域，用于保存数据。IO 设施通常将输入（或输出）数据保存在一个缓冲区中，读写缓冲区的动作与程序中的动作是无关的。我们可以显式地刷新输出缓冲，以便强制将缓冲区中的数据写入输出设备。默认情况下，读 cin 会刷新 cout；程序非正常终止时也会刷新 cout。

内置类型 (built-in type) 由语言定义的类型，如 int。

Cerr 一个 ostream 对象，关联到标准错误，通常写入到与标准输出相同的设备。默认情况下，写到 cerr 的数据是不缓冲的。cerr 通常用于输出错误信息或其他不属于程序正常逻辑的输出内容。

字符串字面值常量 (character string literal) 术语 string literal 的另一种叫法。

cin 一个 istream 对象，用来从标准输入读取数据。

类 (class) 一种用于定义自己的数据结构及其相关操作的机制。类是 C++ 中最基本的特性之一。标准库类型中，如 istream 和 ostream 都是类。

类类型 (class type) 类定义的类型。类名即为类型名。

clog 一个 ostream 对象，关联到标准错误。默认情况下，写到 clog 的数据是被缓冲的。clog 通常用于报告程序的执行信息，存入一个日志文件中。

注释 (comment) 被编译器忽略的程序文本。C++ 有两种类型的注释：单行注释和界定符对注释。单行注释以 // 开始，从 // 到行尾的所有内容都是注释。界定符对注释以 /* 开始，其后的所有内容都是注释，直至遇到 */ 为止。

条件 (condition) 求值结果为真或假的表达式。通常用值 0 表示假，用非零值表示真。

cout 一个 ostream 对象，用于将数据写入标准输出。通常用于程序的正常输出内容。

花括号 (curly brace) 花括号用于划定程序块边界。左花括号 ({) 为程序块开始，右花括号 (}) 为结束。

数据结构 (data structure) 数据及其上所允许的操作的一种逻辑组合。

编辑-编译-调试 (edit-compile-debug) 使程序能正确执行的开发过程。

文件结束符 (end-of-file) 系统特定的标识，指出文件中无更多数据了。

表达式 (expression) 最小的计算单元。一个表达式包含一个或多个运算对象，通常还包含一个或多个运算符。表达式求值会产生一个结果。例如，假设 i 和 j 是 int 对象，则 i+j 是一个表达式，它产生两个

27